Evaluation of Worst Case Execution Time Analysis Methods for Real-Time Systems

Umut Karakaya
Hamm-Lippstadt University
of Applied Sciences (HSHL)
Lippstadt, Germany
umut-faruk.karakaya@stud.hshl.de

***Abstract*—Finding the Worst-Case Execution Time (WCET) of software tasks is a important requirement in the design of safety critical real-time systems. A reliable WCET estimate guarantees that tasks complete before their deadlines and prevents some potential errors. Several analysis methods have been developed in the last years: static, measurement based and hybrid. They rely on different kind of assumptions about the hardware and software analysis. This paper provides a structured overview of these methods, introduces the formal foundations of WCET analysis and discusses the key challenges that affect their trustworthiness in practice.**

I. Introduction

Real time systems are systems in which the correctness of computation depends on both the logical result and the time at which this result becomes available. [1] Examples can be found in automotive control systems, avionics, railway applications and other safety related embedded systems. In these domains, a delayed reaction can be problematic eventhough if the computed value is actually correct. Systems like a brake controller or a flight control function must therefore be analysed with respect to its computed result as well as its completion time.

A central concept for this type of timing analysis is the Worst Case Execution Time (WCET). The WCET of a task describes an upper bound on the maximum time the task may need to execute on a given hardware platform. If such a bound is known, it can be compared with the deadline of the task and used as input for schedulability analysis. In this sense, WCET analysis supports the validation of timing guarantees before a real time system is put into operation. Finding a reliable WCET bound is difficult in practice. Modern processors include hardware features such as caches, pipelines, branch prediction, buses and memory hierarchies. These features are useful for improving average performance but at the same time they also make execution times harder to predict. Small differences in input data, memory layout or processor state leads to different execution times. [2] For safety related systems, it means that an observed execution time is not automatically a safe upper bound. The main issue is not the computation of a WCET estimate. It is also necessary to explain why this estimate is safe enough to use for the analysed system. In another saying, the estimate should be higher than the execution time that can occur during operation and the analysis should be based on assumptions that would actually fit the target hardware and software. A static timing analysis depends most of the time on hardware timing models and program flow information . A measurement based analysis depends on test inputs, execution time observations and assumptions about how close the highest observed execution time is to the real worst case. Hybrid and probabilistic methods try to reduce some of these limitations but they also have some additional assumptions and practical challenges.

This paper discusses WCET analysis methods with a focus on their practical tradeoffs and their reliablity. First, the basic terminology of WCET analysis is explained. Then static, measurement based, hybrid and a probabilistic approach is described and compared. The discussion focuses on the main issues that affect confidence in WCET estimates, such as hardware modelling, program path information, measurement coverage and multicore interference. Finally, a small implementation example is used to explain that the choice of a WCET analysis method depends on the target system, the required confidence level and the analysis effort.

II. Background and Formal Foundations

A. Real-Time Systems and Timing Constraints

A real-time system is one in which the correctness of a computation depends on both its logical result and the time at which that result is produced. Each task in such a system is associated with a deadline: a time limit by which execution must be complete. If a task overruns its deadline, the system is said to have experienced a timing failure. In hard real-time systems, failures like those are considered very dangerous and must be avoided by design. [1]

To verify that a system meets its timing requirements, each task must be assigned a WCET estimate an upper bound on the longest possible execution time of the task on the target hardware. This estimate feeds directly into schedulability analysis where a scheduler determines whether all tasks can meet their deadlines under the given execution model. A WCET estimate that is too optimistic will most likely cause a task to

miss its deadline at runtime, one that is too pessimistic wastes processor capacity and makes a schedule that would normally work seem impossible.

B. Control Flow Graph and WCET

A Control Flow Graph (CFG) is useful for WCET analysis because it shows which execution paths a program can take. In a CFG, the nodes represent basic blocks of code and the directed edges represent possible control transfers between these blocks. Besides the graph structure itself, the execution time of each possible path through the graph is also really important for WCET analysis.

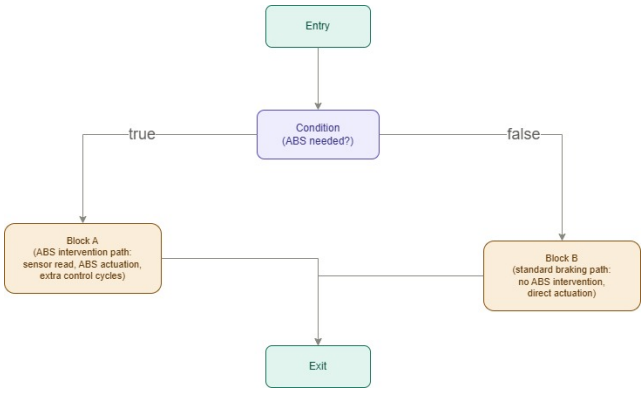

Fig. 1. Simplified Control Flow Graph of an ABS braking function.

The WCET corresponds to the path with the highest total execution time. Since the ABS intervention logic (Block A) performs additional checks and control actions, this path is expected to take more time than the standard braking path (Block B). As a result of that, the path that includes Block A is going to have the highest execution time and can be considered as the worst case path in this simplified example. This shows the basic connection between CFGs and WCET analysis. A CFG helps to see the different execution paths that a program can follow. The timing analysis then has to work on which of these paths is feasible and which one will produce the highest execution time.

Before describing the individual methods, it is useful to explain the differences between deterministic and probabilistic WCET analysis. Deterministic timing analysis aims to compute a single WCET estimate for the analysed task. This estimate is then used as an upper bound in schedulability analysis. Static, measurement based and hybrid methods can all be used in deterministic form. Probabilistic timing analysis follows a different idea. Instead of producing only one deterministic bound. And it computes a probabilistic worst case execution time, called pWCET. A pWCET describes an execution time bound together with a probability that this bound may be exceeded. This is useful when the timing behavior of the platform can be treated statistically [7], still it also requires additional assumptions about measurements and hardware behaviour.

Based on this distinction, WCET analysis methods can be grouped into six variants which are static deterministic, measurement based deterministic, hybrid deterministic, static probabilistic, measurement based probabilistic and hybrid probabilistic analysis methods. [4] This paper focuses mainly on the deterministic methods.

III. WCET Analysis Methods

A. Static Analysis

Static WCET analysis estimates the execution time of a program without running it on the target platform. Instead of collecting measurements, it analyses the program structure and combines this information with a timing model of the hardware. [6] The program structure is often represented with a Control Flow Graph, where the possible execution paths of the program are visible. The hardware timing model is then used to estimate how long instructions or basic blocks can take on the processor.

Static analysis tries to work on the feasible execution paths of a program before it is executed. Using the control flow information and timing model, the analysis aims to determine which feasible path could result in the longest execution time.

The strength of static analysis is that it does not require the worst case path to be observed during testing. In principle, it can examine all possible program paths by using the program structure and a hardware timing model. This makes it attractive for safety critical systems, where relying only on observed test executions may be insufficient.

The weakness is that static analysis needs reliable input information. The hardware timing model must describe the processor behavior accurately. The analysis also needs correct program flow information for example like loop bounds, possible branch targets and infeasible paths. If these assumptions are wrong, too optimistic, too pessimistic or unclear, the computed WCET estimate can also become unreliable. This is especially difficult on modern processors with caches, pipelines and shared hardware resources, because their timing behavior is harder to model precisely.

A common technique used in static WCET analysis is the Implicit Path Enumeration Technique (IPET). IPET avoids explicitly listing all possible execution paths. This is pretty useful because the number of possible paths can grow very quickly when a program contains several branches or loops. Instead of enumerating every path, IPET represents the program structure by integer variables and flow constraints derived from the Control Flow Graph. [5]

The resulting WCET computation is commonly formulated as an Integer Linear Programming (ILP) problem. In this formulation, each basic block (i) has an execution cost (c_i), and (x_i) represents how many times this block is executed. The objective is to maximize the total execution cost [3]:

$$\max \sum_{i=1}^{N} c_i x_i \tag{1}$$

The variables (x_i) are restricted by constraints from the CFG, loop bounds and infeasible path information. Therefore, the solver of this cfg does not need all paths to be listed

manually. A manual listing of all the paths would be too complicated and would require too much effort. It searches for the feasible combination of basic block executions that gives the highest total cost.

For the simplified ABS example the entry block, condition block and exit block are executed once. After the condition block, the program can either execute the ABS intervention block or the normal braking block. This branch can be represented by the constraint:

$$xABS + xNormal = 1 \tag{2}$$

This means only one of the two alternatives is taken in one execution. If the ABS intervention block has a higher timing cost than the normal braking block, then the optimization selects the ABS path as the WCET path.

A small Python implementation shows this idea by using an ILP solver. implementation assigns timing costs to the basic blocks of the ABS CFG, adds the flow constraints and lets the solver maximize the total timing cost. This demonstration is not a complete industrial WCET analyzer because it uses predefined block costs and a very small CFG. Nevertheless, it shows the main idea of static WCET computation with IPET. Each basic block is assigned an assumed timing cost. The Entry block is the start of the function and is assumed to take 1 ms. The Condition block checks whether ABS intervention is required and is assumed to take 2 ms. After this check the program follows either Block A or Block B. Block A represents the ABS intervention logic and is assumed to take 20 ms while Block B represents the normal braking logic and is assumed to take 5 ms. Finally, the Exit block is reached in both cases and is assumed to take 1 ms.

Listing 1. Simplified IPET-based WCET calculation.

```
costs = {
   "Entry": 1,
   "Condition": 2,
   "ABS": 20,
   "Normal": 5,
   "Exit": 1
}

problem += lpSum(costs[b] * x[b] for b in costs)

problem += x["Entry"] == 1
problem += x["Condition"] == 1
problem += x["Exit"] == 1
problem += x["ABS"] + x["Normal"] == 1
```

TABLE I
TIMING COSTS AND SELECTED BLOCKS IN THE SIMPLIFIED ABS EXAMPLE

Block	Cost (ms)	Selected x_i
Entry	1	1
Condition	2	1
ABS Intervention	20	1
Normal Braking	5	0
Exit	1	1

The solver selects the ABS intervention path, as shown in Table I. Therefore, the simplified static WCET estimate is:

$$WCET = 1 + 2 + 20 + 1 = 24 \text{ ms} \tag{3}$$

Static WCET analysis can also be implemented in practical tools although these tools are much more complex than the simplified Python example used in this paper. Tools like aiT and OTAWA apply static analysis to program code or binary executables in order to compute WCET bounds. Static WCET tools usually reconstruct the program structure, analyse possible execution paths, use processor timing models and solve path analysis problems. [8]

B. Measurement Based Analysis

Measurement based WCET analysis estimates the execution time of a program by running it on the target hardware and collecting timing measurements. Instead of building a complete timing model of the processor, this approach observes the behavior of the real system. The highest execution time observed during the test runs is called the Highest Observed Execution Time (HOET) .For the ABS braking example, the braking function can be executed with different input situations like normal braking, strong braking or a road condition where ABS intervention is required. Each test run produces an execution time measurement. If the ABS intervention case produces the highest measured value this value becomes the HOET for the performed tests. This makes the method easy to understand and apply, it is directly based on observed execution times.

The main strength of measurement based analysis is its practical applicability. Since the program is executed on the real hardware, the measurements already include many timing effects of the processor. This is useful when the hardware is too complex to model precisely. Because of that measurement based approaches are common in industrial practice, especially when a detailed static timing model is not available or would be too expensive to build.

The main weakness is that measurements only cover the executions that were actually tested. The true worst case may occur for an input, program path or hardware state that was not observed during testing. Therefore, the HOET is not automatically a safe WCET bound [6]. Some approaches add a safety margin to the HOET, but the choice of such a margin is difficult to justify in a strict scientific way. The trustworthiness of the result depends strongly on the quality of the test inputs, the measurement setup and the coverage of relevant execution scenarios.

A strict interpretation of the measurements is important. After a measurement, the only directly supported result is that the real WCET is greater than or equal to the Highest Observed Execution Time. The distance between the HOET and the real WCET is still unknown and hard to quantify. This distance depends on the quality of the test inputs, the coverage of relevant paths and the hardware states that occurred during the tests.

C. Hybrid Analysis

Hybrid WCET analysis combines ideas from static analysis and measurement based analysis. The program is still executed on the target hardware and timing measurements are collected. However, the analysis does not only rely on complete test runs. It uses structural information about the program such as the Control Flow Graph and possible execution paths. [8]

The main idea is to use measurements for parts of the program and then combine these measurements with control flow information. For example, basic blocks or smaller program segments can be measured during execution. The CFG can then be used to understand how these parts may be connected in different paths. In this way hybrid analysis tries to reason about more than only the exact executions that were observed during testing. This can make hybrid analysis more informative than a purely measurement based approach. Measurement based analysis may miss a path if no test case activates it. Hybrid analysis reduces this problem by using the program structure to identify relevant paths and to guide the analysis. At the same time, it does not require a complete and very detailed hardware timing model in the same way as static analysis. This makes it useful when the hardware is too complex for a precise static model but when pure measurement based evidence is not considered strong enough.

The limitation is that hybrid analysis still depends on the quality of the measurements and the correctness of the program structure information. Combining measured parts is not always simple because of the timing of one block may depend on the previous execution history. Cache state, pipeline behaviour and memory accesses can change when blocks are executed in a different context. Therefore, a measured time for a program part does not always represent its timing in every possible full path.

D. Method Comparison

The three WCET analysis methods differ mainly in the type of information they use and in the confidence they can provide. This table summarizes the main differences between them.

Method	Main input	Main strength	Main limitation
Static	CFG, flow facts, hardware timing model	Strong timing argument if assumptions are valid	Difficult on complex hardware
Measurement based	Execution time measurements	Practical and based on real hardware	Worst case may not be tested
Hybrid	Measurements and control flow information	Combines real data with program structure	Still depends on measurements and assumptions

Fig. 2. Comparison of WCET analysis methods.

The comparison shows that there is no universally best WCET analysis method. For this reason, the choice of method depends on the required confidence level and the complexity of the hardware.

IV. Trustworthiness and Discussion

The trustworthiness of a WCET estimate depends on the assumptions behind the analysis besides the selected method. Static, measurement based, hybrid and probabilistic approaches all require reliable information about the program and the target platform. This includes hardware timing models, loop bounds, infeasible paths, representative test inputs, measurement quality and platform assumptions. If these inputs are incomplete or too optimistic, the final WCET estimate can become unsafe. [4]

The suitable method depends on the system context. Static analysis can provide strong timing arguments when the hardware and program flow are predictable enough to model. Measurement based analysis is practical on complex processors but its confidence depends on how well the tests cover relevant paths and hardware states. Hybrid analysis combines measurements with control flow information while probabilistic methods use pWCET estimates based on statistical assumptions. None of these approaches removes the trustworthiness problem completely. [6] In safety critical real time systems, pessimism is usually safer than optimism. Underestimating the WCET can lead to missed deadlines, which may have serious consequences in railway, avionics or any safety related critical real time systems. Pessimistic assumptions help to avoid unsafe underestimation when timing effects are uncertain. However, excessive pessimism can waste processor capacity and lead to unnecessary resource usage. Therefore, a useful WCET estimate should be safe but not unnecessarily high.

V. Conclusion

WCET analysis is essential for validating timing guarantees in real time systems. Static, measurement based, hybrid and probabilistic methods all try to estimate safe execution time bounds but they use different types of information and rely on different assumptions. Static analysis can provide strong timing arguments however it depends on accurate hardware timing models and correct program flow information. Measurement based analysis is practical and uses real hardware data but the Highest Observed Execution Time is not automatically a safe WCET. Hybrid analysis combines measurements with program structure but it does not remove all uncertainty. Probabilistic methods extend the deterministic view with pWCET estimates, at the same time they also depend on representative measurements and valid statistical assumptions.

The main result of this discussion is that WCET trustworthiness is not guaranteed by a single perfect method, a method like that doesn't exist. It depends on the quality of models,how it matches the hardware and software being used, measurements and the user. A method can be theoretically useful and the result can still become unreliable if the required input information does not match the real system. Therefore, a WCET estimate should always be interpreted together with the assumptions under which it was obtained.

In safety critical real-time systems, the selected WCET method must match the target system, its safety requirements, its hardware complexity and the available analysis effort. A

pessimistic estimate is usually safer than an optimistic one because underestimating the WCET most likely causes missed deadlines during operation. At the same time, over pessimism can waste processor capacity and lead to unnecessary resource usage. Becayuse of that, the goal should be computing a bound that is safe, balanced and not unnecessarily high.

REFERENCES

[1] G. C. Buttazzo, *Hard Real-Time Computing Systems: Predictable Scheduling Algorithms and Applications*. Boston, MA, USA: Springer US, 1997.

[2] Z. Xuewen, J. Jiang, L. Ting, and H. Xing, "Optimization of Multi-core Interconnection for WCET Analysis in Automotive Application," in *Proc. 2012 Fourth International Conference on Computational Intelligence and Communication Networks*, Mathura, India, 2012, pp. 483–486, doi: 10.1109/CICN.2012.155.

[3] B. Lisper and M. Santos, "Model Identification for WCET Analysis," 2009 15th IEEE Real-Time and Embedded Technology and Applications Symposium, San Francisco, CA, USA, 2009, pp. 55-64, doi: 10.1109/RTAS.2009.16.

[4] J. Abella et al., "WCET analysis methods: Pitfalls and challenges on their trustworthiness," 10th IEEE International Symposium on Industrial Embedded Systems (SIES), Siegen, Germany, 2015, pp. 1-10, doi: 10.1109/SIES.2015.7185039.

[5] Yau-Tsun Steven Li and Sharad Malik. 1995. Performance analysis of embedded software using implicit path enumeration. In Proceedings of the ACM SIGPLAN 1995 workshop on Languages, compilers, & tools for real-time systems (LCTES '95). Association for Computing Machinery, New York, NY, USA, 88–98.

[6] Wilhelm, R., Engblom, J., Ermedahl, A., Holsti, N., Thesing, S., Whalley, D., ... & Stenström, P. (2008). The worst-case execution-time problem—overview of methods and survey of tools. ACM transactions on embedded computing systems (TECS), 7(3), 1-53.

[7] J. Abella, D. Hardy, I. Puaut, E. Quiñones and F. J. Cazorla, "On the Comparison of Deterministic and Probabilistic WCET Estimation Techniques," 2014 26th Euromicro Conference on Real-Time Systems, Madrid, Spain, 2014, pp. 266-275, doi: 10.1109/ECRTS.2014.16.

[8] J. Gustafsson, "Usability Aspects of WCET Analysis," 2008 11th IEEE International Symposium on Object and Component-Oriented Real-Time Distributed Computing (ISORC), Orlando, FL, USA, 2008, pp. 346-352, doi: 10.1109/ISORC.2008.55.